

1. Algorithm using Backtracking approach.

Introduction :-

Backtracking is a recursive strategy that builds a solution piece by piece and removes any path that fails to satisfy the problem's constraints.

Objectives :-

To efficiently find a valid solution by 'pruning' search paths that cannot lead to a successful result.

Theory :-

It treats the problem as a State-space Tree. The algorithm performs a Depth-First-Search (DFS). If a branch reaches a 'dead end', the algorithm retreats (backtracks) to the previous node to try a different path.

Algorithm :-

1. start at the initial state.
2. make a choice from available options.
3. check if the choice is valid (safe).
4. Recurse to the next step if valid.
5. Undo the choice (Backtrack) if it leads to a dead end.

Code :- N-Queen

```
#include <stdio.h>
#include <stdbool.h>
#define N 4

bool isSafe(int b[N][N], int r, int c) {
    for (int i = 0; i < c; i++) if (b[r][i]) return false;
    for (int i = r; j = c; i >= 0 && j >= 0; i--, j--)
        if (b[i][j])
            return false;
    for (int i = r, j = c; j <= N; i++, j--)
        if (b[i][j])
            return false;
    return true;
}

bool solve(int b[N][N], int c) {
    if (c >= N)
        return true;
}
```

```

for (int i = 0; i < N; i++) {
    if (isSafe(b, i, c)) {
        b[i][c] = 1;
        if (solve(b, c + 1))
            return true;
        b[i][c] = 0;
    }
}

```

```

return false;
}

```

```

int main() {
    int b[N][N] = {0};
    if (solve(b, 0))
        for (int i = 0; i < N; i++) printf("\n")
            for (int j = 0; j < N; j++)
                printf("%.1c", b[i][j] ? 'Q' : '.');
    return 0;
}

```

Output :-

```

. . Q .
Q . . .
. . . Q
. Q . .

```

Time complexity :-

• worst case : $O(N!)$ or $O(k^N)$ where N is the depth & k is the choice factor.

Conclusion :-

Backtracking is an optimal brute-force method that saves time by abandoning invalid paths early.

References :-

- Algorithm Design by John Kleinberg.

2. Implement approximation Algorithm.

Introduction:

For many optimization problems (like the Travelling Salesperson Problem or Vertex Cover), no known polynomial-time algorithm exists to find the perfect answer. Approximation Algorithms trade perfection for speed, providing a 'near optimal' solution in polynomial time.

Objective :-

- To solve NP-hard problems in polynomial time.
- To provide a performance guarantee (Approximation Ratio)
- To find a feasible solution when an exact one is too expensive.

Theory :- The Approximate Ratio

An algorithm has an approximation ratio $p(n)$ if for any input of size n , the cost C of the solution produced by the algorithm is within a factor $p(n)$ of the cost C^* of an optimal solution:

$$\max \left(\frac{C}{C^*}, \frac{C^*}{C} \right) \leq p(n)$$

Algorithm :- Vertex cover (2-Approximation)

A vertex cover is a set of vertices such that every edge in the graph is incident to at least one vertex in the set.

1. Initialize an empty set vc .
2. While there are edges remaining in the graph.
 - Pick an arbitrary edge (u, v)
 - Add both u and v to vc .
 - Remove all edges incident to u or v from the graph
3. Return vc .

Code :- 2 approximation heuristic for the Vertex cover problem.

```
#include <stdio.h>
#include <stdbool.h>

#define VS // No. of vertex

void approximateVertexCover(int adj[V][V]) {
    bool visited[V] = {false};
    for (int u = 0; u < V; u++) {
        for (int v = 0; v < V; v++) {
```

```

    if (adj[u][v] == 1 && !visited[u] && !visited[v]) {
        visited[u] = true;
        visited[v] = true;
    }
}

printf("Approximate vertex cover: ");
for (int i = 0; i < V; i++) {
    if (visited[i])
        printf("%d ", i);
}
printf("\n");

int main() {
    int adj[V][V] = {
        {0, 1, 1, 1, 0},
        {1, 0, 0, 0, 0},
        {1, 0, 0, 0, 0},
        {1, 0, 0, 0, 1},
        {0, 0, 0, 1, 0}
    };
    ApproximateVertexCover(adj);
    return 0;
}

```

Output :-

Approximate vertex cover: 0 1 3 4

Time complexity :-

- Time: $O(V+E)$, where V is vertices and E is edges. This is much faster than the $O(2^V)$ required for an exact solution.
- Approximation Ratio: 2

Conclusion :-

Approximation algorithms are essential for real-world computing. While they don't give the 'best' answer, they provide a 'good' answer quickly with a proven boundary on how far off they might be.

Reference :-

- Vazirani, Vijay V. (2001). 'Approximation Algorithms'
- ~~Cormen~~ Cormen et al. 'Introduction to Algorithms'.